

SFC-RG 全体講義

プロトコル設計における脅威の明確化と正当性証明

moz@sfc.keio.ac.jp

April 30, 2025

主題

- ・ プロトコルの設計について紹介する
- ・ 脅威モデルについて紹介する
- ・ プロトコルの正当性証明 (Formal Proof) について紹介する
- ・ 最後に Delight 的な (分散システムとデジタルアイデンティティについての) 研究課題を紹介する

目的

- ・ 分散システムにおけるプロトコルの設計 (のお気持ち) を理解する
- ・ プロトコルの設計における正当性保証の考え方 (のお気持ち) を理解する
- ・ 自身のプロトコルを学術論文としてまとめる際、どのような構成で何を明らかにすれば良いかを理解する

プロトコルとは？

プロトコルの定義 (Oxford Dictionary)

A set of rules governing the exchange or transmission of data between entities.

エンティティ：プロトコルに参加する個々の主体（ユーザー、デバイス）を指す。

アルゴリズムの定義 (Oxford Dictionary)

A set of instructions for performing a task or solving a problem.

アルゴリズムは計算手順に焦点を当てるのに対し、プロトコルは参加者間の相互作用に重点を置く。¹一般的なプロトコルの例：HTTP、SMTP、TLS/SSL、SSH、OAuth、Paxos、Raft

¹プロトコルは英語や日本語など共通のルールで会話する例として説明されることが多いが、これは相互作用の規則を表しているためである。一方で「日本語アルゴリズム」という表現が一般的でないのは、アルゴリズムが相互作用ではなく計算手順を表すためである。

正当 (Correctness) なプロトコルの設計をどうやるか

この講義はプロトコルの設計手法については紹介しない。

皆さんが自分のドメインで設計したプロトコルがなぜ正当 (Correctness) と言えるのかをどのように他人に説明できるかを考える講義

プロトコルの正当性を示すには、以下の点を明確にすることが重要である：

- ・ どのようなエンティティがプロトコルに関与するか
- ・ エンティティ間でどのような情報交換が行われるか
- ・ プロトコルはどのような条件を満たせば安全であると言えるのか
- ・ それぞれのエンティティがどのような振る舞いをする可能性があるか
- ・ そのようなエンティティが存在するときに、プロトコルが正当²であるかどうかをどのように判断するか

²Safety (悪いことが起こらない) , Liveness (良いことがいつか起こる) のプロパティを共に満たすか

敵対者と脅威モデル

プロトコル設計において、どのような振る舞いを許容するかを考える上では、エンティティの能力と行動に関する仮定を明確にすることが重要である。

敵対者

An adversary refers to an attacker, typically with malicious intent, who undertakes actions against a system or protocol

敵対者は、プロトコルに参加するエンティティを制御下に置き、そのエンティティの動作を変更または監視できる状態。

脅威モデル

脅威モデル (threat model) とは、敵対者の能力と行動に関する仮定を形式化したモデル。

敵対者と脅威モデル

敵対者は、正直なエンティティを制御し、プロトコルに腐敗したエンティティを導入する。

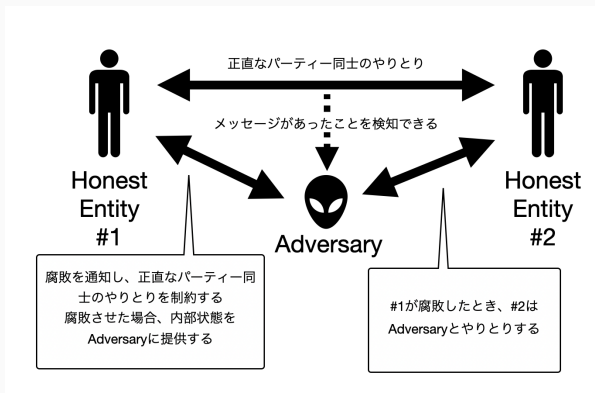


Figure 1: 敵対者とエンティティの関係。敵対者は正直なエンティティのメッセージを傍受したり、直接エンティティを制御したりすることで、プロトコルに腐敗したエンティティを導入することができる。⁴

³ エンティティが直接、腐敗することはないことに注意！

脅威モデルの構成要素

脅威モデルは以下の5つの要素から構成から構成される：

1. Corruption Model. 敵対者が制御するエンティティの行動様式
2. Computational Power. 敵対者が利用できる計算リソースの制約
3. Adaptivity. 敵対者がエンティティを腐敗させるタイミングと方法
4. Visibility. 敵対者が観察できる情報の範囲
5. Mobility. 敵対者によるエンティティの腐敗と回復のパターン

この要素の組み合わせ方によって、異なる脅威モデルが定義される。どのような要素を選択するかは、プロトコルの目的、実行されうる環境によって決まる。

腐敗モデル (Corruption Models)

腐敗モデルは、敵対者はエンティティをどのように制御できるかを定義する。

- ・ Passive. プロトコルに完全に従いながら、観測したすべての情報を蓄積して活用できる。しかし、プロトコルの仕様に逸脱することはできない。⁵⁶
- ・ Crash. 応答を任意のタイミングで停止できるが、停止時間外は正確にプロトコルに従って動作する。
- ・ Omission. エンティティは受信したのメッセージを任意に削除または無視できる。送信するメッセージの内容を変更することはできない。
- ・ Byzantine. エンティティはプロトコル仕様から任意に逸脱できる。

各モデルは前のモデルを包含している：Byzantine $\supset$ Omission $\supset$ Crash $\supset$ Passive

⁵具体例として銀行の従業員、顧客の取引データを正しく処理しながらも、そのデータをこっそり記録して個人的に利用する場合。

⁶Honest-but-Curious, Semi-Honest モデルとも呼ばれる。

適応性モデル (Adaptivity Models)

適応性モデルは、敵対者がいつ、どのように参加者を腐敗させるかを定義する：

- ・ Static：敵対者は腐敗させるエンティティをプロトコル実行前に選ぶ。実行中に変更できない。
- ・ **Adaptive**：敵対者はプロトコル実行中に腐敗させるエンティティを動的に選択できる。

エンティティが少ない場合、プロトコルの通信フェーズ数が多い場合、プロトコルの実行時間が長い場合、Adaptive モデルを考慮する。例えば、鍵交換モデルなどでは、Static モデルを考えることが一般的。コンセンサスアルゴリズムでは、Adaptive モデルを考えることが一般的。

可視性モデル (Visibility Models)

可視性モデルは、敵対者がどの程度のメッセージや内部状態を観察できるかを定義する。

- ・ Full Information. 敵対者はすべてのメッセージ内容と各エンティティの内部状態（実行中の変数）を完全に観察可能
- ・ Private Channels. 正直なエンティティ間の通信内容は敵対者から隠されるが、通信の発生事実やメタデータ（送信元、宛先、タイミング、メッセージサイズ等）は観察可能。腐敗したエンティティの状態、メッセージ内容は観察可能。
- ・ Erasure Model. Private Channels に対して、参加者が腐敗する前に消去されたデータは、敵対者は観察・復元できない制約を足したもの。
- ・ Oblivious. メッセージのヘッダー情報（送信元、宛先、長さ）のみを観察可能で、内容は完全に隠蔽される。⁷

暗号プロトコルでは、機密性 (intended to be kept secret) を保証するために、Private Channels モデルを考えることが多い。

⁷Trusted Computing のようなシステムでは、このモデルを考えることが多い。

計算能力モデル (Computational Power Models)

計算能力モデルは、敵対者の計算資源の制限を定義する：

- ・ Unbounded. 無制限の計算資源を持つ敵対者。理論上はすべての暗号を破ることができる。
- ・ Polynomially Bounded. 多項式時間の計算⁸に制限される敵対者
- ・ Fine-grained. 特定の計算制限を持つ敵対者。
秒間ハッシュ計算回数に制限があるモデル (Hashrate)

重要なのは、エンティティの計算能力ではなく、敵対者の計算能力である。敵対者は Visibility Models にそって、エンティティのメッセージや内部状態を観察できる。その情報を敵対者はどのような計算資源を用いて活用できるかを定義する。

⁸ある問題 (Problem) を解くときに、その問題の解を求める計算複雑性がオーダー $O(n^c)$ $n, c \in \mathbb{Z}$ で表せる。
例えば、離散対数問題、素因数分解問題の難しさなど。

一般的には Probabilistic Polynomial Time (PPT) が確率チューリングマシンにおいて、入力となるセキュリティパラメータ $\lambda \in \mathbb{Z}$ (鍵長など) の大きさに対して、乱数を用いながらも 多項式時間 λ^c 以内に必ず終了するアルゴリズム

移動性モデル (Mobility Models)

移動性モデルは、敵対者による参加者の腐敗と回復のパターンを定義する：

- ・ Fixed. 一度腐敗したエンティティはプロトコル実行中ずっと腐敗したままであり、回復することはない
- ・ Mobile. 敵対者はエンティティを動的に腐敗させたり、回復させたりできる。ただし、任意の時点で腐敗しているエンティティの総数には上限がある⁹

IoT やモバイルデバイス、システムの長期運用の視点を考えると、一般的に Mobile モデルを考える。

⁹Adaptivity Models の Adaptive との混同に注意すること。Adaptive は敵対者がエンティティを腐敗させるタイミングを選択できることを指す。Mobility はその条件下において、一度腐敗させたエンティティが正常なエンティティとして回復するかどうかを考慮している。

適切な仮定を置くことの重要性

セキュリティプロトコルの研究において、**仮定の明確化**は極めて重要である。

- ・ 仮定が弱すぎる場合. 現実的な攻撃に対して脆弱なプロトコルになる可能性がある
- ・ 仮定が強すぎる場合. 設計や実装が複雑になり、その影響でパフォーマンスが下がり、実用性が低下する可能性がある

仮定を明示する価値：

- ・ **プロトコルの正当性**を正確に理解できる
- ・ 異なるプロトコルを同じ基準で公正に比較できる

実際のシステム設計では、特定の用途や発生しうるリスクに基づいて適切な仮定（脅威モデル）を選択することが重要である。

例：Practical Byzantine Fault Tolerance (PBFT)

PBFT

- ・ PBFT は、Byzantine 障害を許容しつつも、Linearizable¹⁰を達成するレプリケーションプロトコル¹¹である。
- ・ ただし、 $n \geq 3f + 1$ （ここで f は敵対者の数、 n はレプリカの総数）のときに限る。

PBFT プロトコルは、非同期通信モデルにおいて初めて Linearizable を達成することを示したプロトコルである。発表から 20 年経った現在でも、PBFT は依然として広く使われている。

¹⁰Linearizable に複製するとは、すべてのレプリカが同じ順序でクライアント要求を処理し、その反映が書き込みリクエストのセッション内で必ず行われる性質である。各複製データにはシーケンス番号が割り当てられ、このシーケンス番号に従って処理される。

¹¹チューリング完全な状態マシンでは、複製されるデータをオペレーション（状態遷移関数）と見なす。オペレーション f, g がある場合、一般に $f \circ g \neq g \circ f$ であるため、実行順序が重要になる。

PBFT のプロトコル詳細 (1)

PBFT の主なフェーズ：

1. Pre-prepare：プライマリ（リーダー） p がクライアント要求 r にシーケンス番号 s を割り当て、 $\langle \text{PRE-PREPARE}, s, r \rangle$ メッセージを他のレプリカに送信
2. Prepare：レプリカが要求の順序に合意し、 $\langle \text{PREPARE}, s, r \rangle$ メッセージを全レプリカに送信
3. Commit：少なくとも $2f + 1$ の PREPARE メッセージを受信したレプリカは $\langle \text{COMMIT}, s, r \rangle$ メッセージを送信し、実行を確約
4. Reply： $2f + 1$ の COMMIT メッセージを受信したレプリカは要求を実行し、結果をクライアントに返送

PBFT のプロトコル詳細 (2)

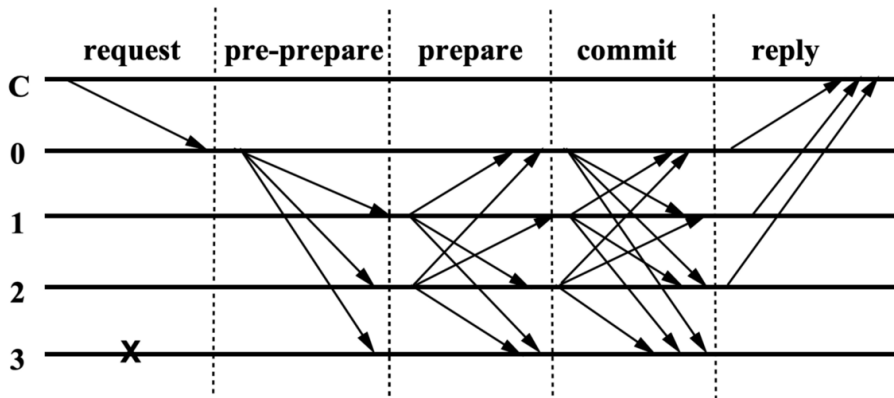


Figure 2: PBFT のシーケンシャルダイアグラム

PBFT の脅威モデル

PBFT の脅威モデルを考えてみよう。

PBFT の脅威モデル Λ_{PBFT}

- Corruption : **Byzantine** - レプリカは異なるメッセージを異なるレプリカに送信するなど、任意に振る舞うことができる
- Adaptivity : Adaptive - 敵対者はプロトコルの各フェーズで異なるレプリカを動的に腐敗させられる
- Computational Power : Polynomially Bounded - 多項式時間の計算に制限される（暗号署名が偽造不可能と仮定）
- Visibility : Private channels - 正直なレプリカ間の通信内容は秘密であり、内部状態も敵対者からは見えない
- Mobility : Mobile - 敵対者はプロトコルの各フェーズで異なるレプリカを動的に腐敗させられる

PBFT の正当性保証

$n \geq 3f + 1$ のとき (f は腐敗ノードの数、 n はレプリカの総数) :

- ・ **安全性** (Safety) : 正直なレプリカが同じシーケンス番号で異なる値をコミットすることはない
- ・ **活性** (Liveness) : すべてのクライアント要求は最終的に処理され応答が返される

を満たすことを証明できれば、PBFT は正当 (Correct) であると言える。

すなわち敵対者が脅威モデル Λ_{PBFT} の範囲においてどのような攻撃を行っても、Safety と Liveness を満たすことを証明できれば、PBFT は正当 (Correct) であると言える。

Question: なぜ $n \geq 3f + 1$ のとき Safety が満たされるのか？ PBFT がセキュアであることを説明してみよう！

仮定: 二つの異なる値 v と v' が同じシーケンス番号 s でコミットされたと仮定する。

Lemma

各値がコミットされるためには、少なくとも $2f + 1$ 個のレプリカから *PREPARE* メッセージが必要である。 $(3f + 1$ 個のレプリカのうち、腐敗したレプリカは最大 f 個)

- ・ 値 v がコミットされるには少なくとも $2f + 1$ 個のレプリカ群 $\mathcal{P}$ からの *PREPARE* メッセージが必要
- ・ 同様に、値 v' がコミットされるには少なくとも $2f + 1$ 個のレプリカ群 $\mathcal{Q}$ からの *PREPARE* メッセージが必要
- ・ 合計 $n \leq 3f + 1$ レプリカのうち、悪意あるレプリカは最大 f 個
- ・ $|\mathcal{P} \cap \mathcal{Q}| \geq 2f + 1 + 2f + 1 - n \geq 4f + 2 - (3f + 1) = f + 1$
- ・ つまり、少なくとも $f + 1$ 個の**正直なレプリカ**が両方の値 v と v' について *PREPARE* メッセージを送信したことになる
- ・ しかし、許容する腐敗レプリカは最大で f 個であって、正直なレプリカは同じシーケンス番号に対して複数の値に *PREPARE* を送信しない（プロトコルの定義により）
- ・ これは矛盾であり、したがって仮定は誤り

複雑化したプロトコルの正当性証明

実運用では PBFT のようなシンプルなプロトコルは稀であり、既存のプロトコルを部品として組み合わせた複合的なプロトコルが一般的である。

- ・ 既存のレプリケーション層（例：PBFT）上にアプリケーション層プロトコルを構築することが多い¹²

ハイブリッドプロトコル

n 個のプロトコル $\pi_1, \dots, \pi_n$ を組み合わせて得られるハイブリッドプロトコルを $\Pi[\pi_1, \dots, \pi_n]$ と表記する。

Motivation：複合プロトコル $\Pi[\pi_1, \dots, \pi_n]$ の正当性をどのように示すか？

- ・ 各構成要素を個別に分析して全体の正当性を証明できないか？
- ・ 任意の環境での正当性を保証するには？

¹²例：PBFT のうえにタイムスタンプ機構を作る. kekeho さんの GP2 に関連.

Universal Composability Framework

Universal Composability (UC) Framework を使えば、複雑なプロトコルの正当性を、シンプルな部品の完全性保証を元に示せる。つまり、複合プロトコル $\Pi[\pi_1, \dots, \pi_n]$ が正当であることを証明するには、各構成要素 $\pi_1, \dots, \pi_n$ の正当性を証明し、それらの組み合わせ方が適切であることを示す。

理想機能 $\mathcal{F}$ の定義

理想機能 $\mathcal{F}$ とは、任意の関数 f を実行する計算要素であり、全てのエンティティから入力を受け取り、 f を計算し、各エンティティに出力を返す。すなわち、インターフェースを定義し、任意の入力に対して、出力を返す写像 f を定義する。これは破壊不可能 (すなわち、インターフェースや f の写像を変更することができない) な信頼されたエンティティとして考えることができる。

プロトコル π の定義

プロトコル π とは、実際のプログラムであり、複数のエンティティ間で実行される手続き (プロトコルの具体的な実装) である。これは理想機能 $\mathcal{F}$ を実現するために設計され、メッセージの交換、計算処理、状態の管理などを含む具体的な手続きの集合である。

理想／現実世界パラダイム

UC には理想機能 $\mathcal{F}$ とプロトコル π を実行するために二つの世界がある:

理想世界 (Ideal World)

- ・ 理想機能 $\mathcal{F}$ が計算を行い、敵対者はシミュレータ Sim として振る舞う。
- ・ ダミーエンティティは入力を $\mathcal{F}$ に送り、 $\mathcal{F}$ からの出力を環境 $\mathcal{Z}$ に返す。
- ・ シミュレータ Sim は理想機能 $\mathcal{F}$ とのインターフェースを通じて、現実世界の敵対者の行動を模倣し、その結果を環境に返す。

現実世界 (Real World)

- ・ 現実世界は実プロトコル π とその参加エンティティ、敵対者 $\mathcal{A}$ で構成される。
- ・ エンティティたちはプロトコル π に従って互いに通信しながら計算を進める。
- ・ 敵対者 $\mathcal{A}$ は腐敗したエンティティを制御し、全ての内部状態やアクションが $\mathcal{A}$ によって制御される。
- ・ 環境 $\mathcal{Z}$ はエンティティと敵対者に入力を提供し、全ての出力を収集する。

理想と現実を区別する環境というエンティティ

環境 $\mathcal{Z}$

- ・ 環境 $\mathcal{Z}$ はプロトコル外の全てのシステム（ユーザ，他のプロトコルなど）を抽象化したもの。
- ・ $\mathcal{Z}$ は理想世界ではダミーエンティティと $\mathcal{F}$ およびシミュレータ Sim に、現実世界ではエンティティと敵対者 $\mathcal{A}$ に対して入力を送り、出力を受け取る。
- ・ $\mathcal{Z}$ は両世界の相互作用を観察し、どちらの世界で実行されているかを推測しようとする。¹³

¹³ $\mathcal{Z}$ は現実世界における π と $\mathcal{A}$ との相互作用と、理想世界における $\mathcal{F}$ と Sim との相互作用の違いを探し出す

UC における正当性の主張

UC 正当性の定義

環境 $\mathcal{Z}$ は現実世界と理想世界の相互作用の違いを探そうとするが、どちらも区別できなければ π は理想機能 $\mathcal{F}$ を正当に実現していると言える。¹⁴。

厳密な定義セキュリティパラメータ $\kappa \in \mathbb{N}$ とプロトコル Π について、以下が成り立つとき、 Π は理想機能 $\mathcal{F}$ を “UC-realize” するという：任意の確率的多項式時間¹⁵ 敵対者 $\mathcal{A}$ に対し、確率的多項式時間アルゴリズム Sim が存在し、任意の確率的多項式時間環境 $\mathcal{Z}$ と任意の入力 $x \in \{0, 1\}^*$ について：

¹⁴ これを「 π は $\mathcal{F}$ を UC-realize する」という

¹⁵ UC では確率的多項式時間敵対者に対してのみ成立する

UC を使うと、正当性証明において何が嬉しいのか？

- ・ モジュール性 (Modularity). 複雑な課題を小さなプロトコルに分解でき、各プロトコルの正当性を独立して証明できる。
- ・ 合成定理 (Composition Theorem). プロトコル π が理想機能 $\mathcal{F}$ を UC-realize し、プロトコル ρ が $\mathcal{F}$ を使って理想機能 $\mathcal{G}$ を UC-realize するなら、 π で $\mathcal{F}$ を置き換えたプロトコル $\rho^{\mathcal{F} \rightarrow \pi}$ も $\mathcal{G}$ を UC-realize する。

UC フレームワークの核心は「合成可能性 (Composability)」：一度証明されたプロトコルの正当性は、どのような複雑な環境でも保持される。これにより、安全なプロトコルの段階的な構築と再利用が可能になる。

プロトコルインスタンスとセッション識別

実世界では、同じプロトコルの複数のコピーが異なるユーザーによって同時に実行される。プロトコルはいわば型であり、実際にはその型の実体であるインスタンスが実行される。

プロトコルインスタンス

プロトコル π のインスタンスとは、そのプロトコルの具体的な実行例であり、通常は一意の識別子で区別される。

- ・ セッション識別子 (sid) : 各プロトコル実行を識別する一意の識別子
- ・ たとえば... ブロックチェーン上では、スマートコントラクトのアドレスが *sid* として機能する

UC フレームワークでのインスタンス処理: 基本的な UC フレームワークでは、各プロトコルインスタンスは独自のハイブリッド機能インスタンスを持つ。しかし、これは現実的ではない場合がある（例：同じハッシュ関数が複数のプロトコルで使用される場合）。

一般化された Universal Composability (GUC) Framework

GUC Framework とは

UC の拡張フレームワークで、複数のプロトコルインスタンスで共有される「グローバル機能」の概念を導入している。

グローバル機能 (Global Functionality)

- ・ 複数のプロトコルインスタンスで共有される機能
- ・ 例：ハッシュ関数、デジタル署名、共有台帳など
- ・ すべてのプロトコルセッションで一貫した内部状態を維持

GUC によって、並列実行環境を実現するだけでなく、インスタンスを個別に分析する必要がない。

本当に仮定はこれだけでいいのか？

ここまでで用いた仮定は以下の通りである：

- ・ ネットワークモデル：非同期か同期か、部分同期（通信遅延に上限がある）か
- ・ 敵対者モデル：敵対者はどのように振る舞うか
- ・ 腐敗ノードの数：攻撃者が制御できるエンティティの数の条件 ($n \geq 3f + 1$ など)

Question: 実世界で動かす上で考えるべき脅威、仮定はこれだけか？

これだけではすべてのプロトコルに Sybil Attack が適用できてしまう。

Sybil Attack

単一のエンティティが複数のエンティティを模倣し、システム内で不当な影響力を得る攻撃である。¹⁶

PBFT では $n \geq 3f + 1$ の条件がある。 n はノードの数、 f は敵対者の数である。これは各レプリカが一意に識別化であり、メッセージの送信元に偽りが無いことを暗黙の前提においている敵対者が単一のエンティティ（敵対者）が複数のエンティティとして振る舞えることを許すと、条件が破綻する。^a

^aCorruption Model の Byzantine に一定の妥協を許している。これはいいのか？

¹⁶ 1 人のエンティティが山田花子と山田次郎の二つの役を演じているが、他のエンティティは山田花子と山田次郎が同一のエンティティであるかどうかを判断できない。

デジタル署名

メッセージの送信元に偽りがないことを検証するためには、メッセージにデジタル署名を付与し、デジタル署名を検証する必要がある。

デジタル署名

数学的な関係性がある公開鍵 (verification key) pk と秘密鍵 (signing key) sk のペアを用いて、メッセージの送信元を検証する。

- ・ メッセージ m をデジタル署名する： $\sigma \leftarrow \text{SIGN}(sk, m)$
- ・ メッセージ m と署名 σ を検証する： $\{0, 1\} \leftarrow \text{VERIFY}(pk, m, \sigma)$ where 1 if m is valid, 0 otherwise

具体的なデジタル署名スキームとしては、RSA、ECDSA、EdDSA、Schnorr 署名などがある。

メッセージの真正性の検証

エンティティ $\mathcal{P}$ がメッセージ m をエンティティの $\mathcal{Q}$ に送付したとする。エンティティ $\mathcal{Q}$ はメッセージ m を確かに $\mathcal{P}$ から受け取ったことを検証する必要がある。

- ・ エンティティ $\mathcal{P}$ がメッセージ m に対するデジタル署名 σ を公開鍵 $pk_{\mathcal{P}}$ に対応する秘密鍵 $sk_{\mathcal{P}}$ を用いて生成する。
- ・ $\mathcal{P}$ は (m, σ) を $\mathcal{Q}$ に送付する。
- ・ $\mathcal{Q}$ は受け取った (m, σ) を検証する： $\text{VERIFY}(pk_{\mathcal{P}}, m, \sigma) \stackrel{?}{=} 1$

検証の正当性確保に必要な仮定: $\mathcal{Q}$ は $\mathcal{P}$ の公開鍵 $pk_{\mathcal{P}}$ を知っている。

Question: これだけでよいか？

Answer: 不十分。“デジタル署名で送信元を区別できる $\Leftrightarrow$ エンティティを区別できる”という命題は必ずしも成立しないからである。

デジタル署名はメッセージの送信者を区別できるが、エンティティと公開鍵の一対一対応が仮定されなければ、検証の正当性は保証されない。

- ・ 単一のエンティティが複数の公開鍵を生成し複数のエンティティとして振る舞える
- ・ エンティティと公開鍵の一対一対応を保証するためには Key Management System¹⁷が必要。KMS は一対一対応の検証する¹⁸

¹⁷Key Management System (KMS) は、エンティティと公開鍵の一対一対応を保証するシステムである。Public Key Infrastructure (PKI) は Key Management System の一種である。

¹⁸検証は二つある。第一にあるエンティティが自分は山田花子と主張したが、本当に山田花子であるかどうかを検証する。第二にエンティティ群と公開鍵の写像を定義すると、その関係性が全単射であることを確認する。確認後、KMS は紐付きが有効であることを示す署名 (=証明書) を発行する。利用者は KMS を信頼することで、この検証を署名の検証に置き換えることができる。

しかし、KMS には以下の問題がある：

- ・ 認証局の存在を許す = 信頼できる第三者の存在を仮定する
- ・ 単一障害点とスケーラビリティの問題が生じる

ほかにも Proof-of-Work：新たな識別子の作成に計算資源を要求する研究もある。

- ・ 分散システムの文脈でプロトコルを俯瞰すると、分散システムの魅力や効力を最大限発揮できないボトルネック（暗黙、非自明な仮定）が存在する。それは必ずしも学術論文に書かれているとは限らない。

どうやったらインパクトのある問題を見つけるか？

- ・ 運用を片手に持つ、ということは、理論の中で議論されたプロトコルの解釈方法を多様化できる力があることと等しい。
- ・ すなわちそのプロトコルが置かれる状況（提案手法の論理を支える仮定^a）を変えるとボトルネックが無視できるか、できないかを判断できる。
- ・ そのような探求がインパクトがあり、明確な Problem Statement に役立つと moz は信じている。

^a例えば、敵対者モデル、ネットワーク、タイミングモデル

今回は理論的な視点を提供したが、ぜひ運用視点を他の kG から聞くことで、自身の研究（分野）に新たな解釈をもたらすかもしれない。そのような解釈を皆で有機的に生み出し、共有し、高め合う全体講義になればと願っている。

- <https://www.cl.cam.ac.uk/teaching/2122/ConcDisSys/dist-sys-handout.pdf>
- <https://www.bu.edu/macsf/files/2016/03/2016-03-18-uc-tutorial-slides.pdf>